

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Auditor Scan

#	Scan ID 2	✦	Date September 17, 2026
📦	Organization ensuro	📦	Repository access-managed-proxy
📄	Branch main	📄	Commit Hash 80d7344c...4fd1ed1a

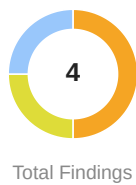
Contracts in scope

contracts/AMPUtils.sol contracts/AccessManagedProxy.sol contracts/AccessManagedProxyBase.sol
contracts/interfaces/IAccessManagedProxy.sol

Code Statistics

🐛	Findings 4	📄	Contracts Scanned 4	☰	Lines of Code 291
---	---------------	---	------------------------	---	----------------------

Findings Summary



- High Risk (0)
- Medium Risk (2)
- Low Risk (1)
- Info (1)
- Best Practices (0)

Code Summary

AccessManagedProxy is an access-controlled upgradeable proxy framework developed by Ensuro that extends the ERC-1967 proxy pattern by integrating OpenZeppelin's `IAccessManager` directly into the call routing mechanism. Instead of relying solely on access control checks defined within the implementation contract logic, the proxy intercepts incoming calls at the delegate layer and validates caller permissions against a centralized `AccessManager` contract prior to forwarding execution.

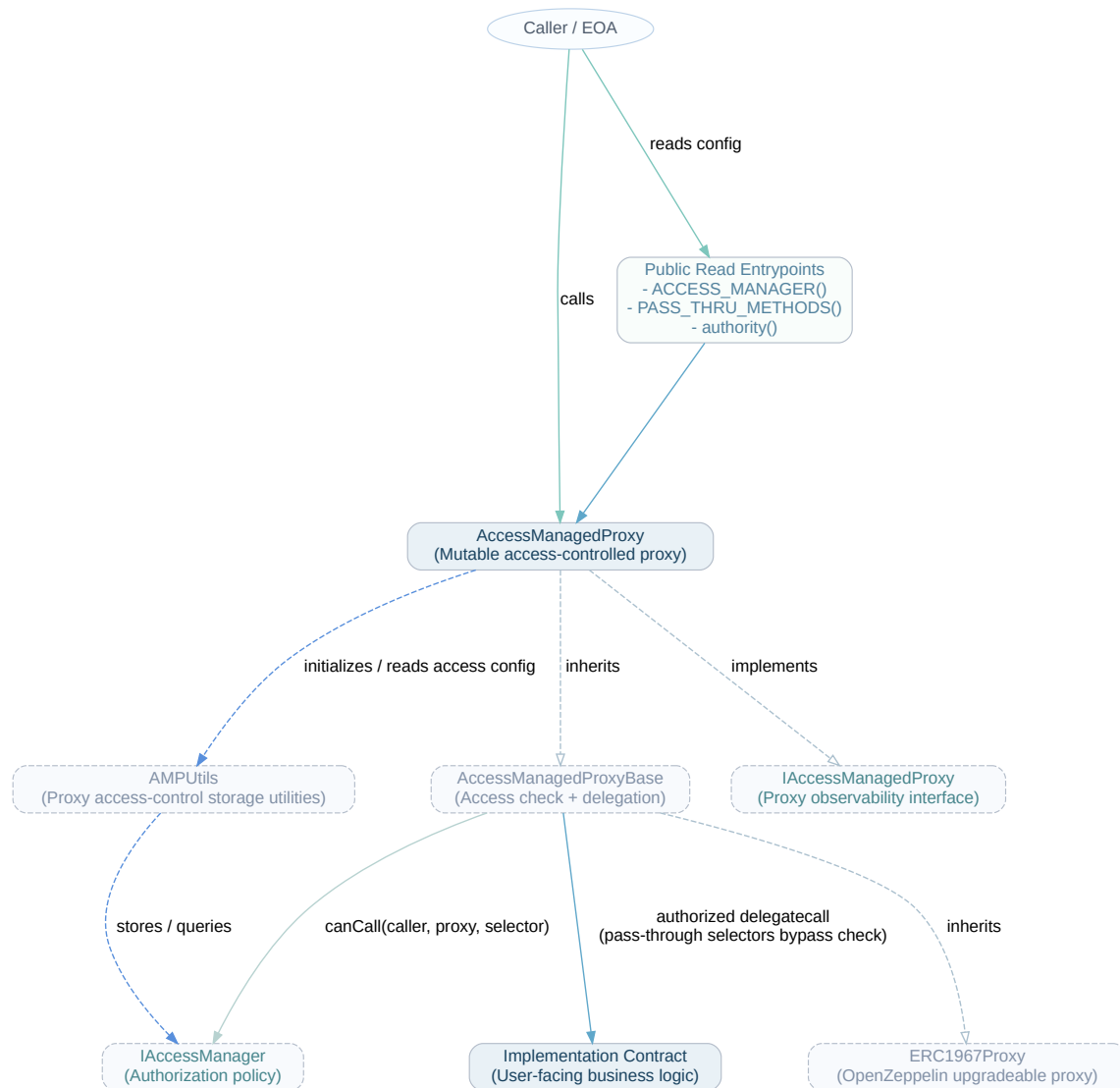
The protocol is composed of three core components: the base proxy contract (`AccessManagedProxyBase`), the mutable deployment implementation (`AccessManagedProxy`), and a dedicated utility library (`AMPUtils`). When an external account or contract executes a transaction or query against the proxy, the internal `_delegate` routing logic inspects the 4-byte function selector (or synthetic selectors for fallback/receive operations). If the function selector is registered under `passThruMethods`, access control verification is bypassed to optimize gas consumption (commonly used for view functions or universally accessible methods). If not bypassed, the proxy executes `canCall(caller, target, selector)` against the configured `IAccessManager`. If authorization is granted immediately, the call is delegated to the implementation contract; otherwise, the transaction reverts with `AccessManagedUnauthorized`.

State storage for the proxy configuration—including the `IAccessManager` address and pass-through selector sets—is managed according to the ERC-7201 namespaced storage standard via `AMPUtils` to eliminate storage collisions with proxy implementation contracts. The proxy itself intentionally exposes no direct administrative mutating endpoints; configuration updates, such as updating the access manager or replacing pass-through method lists, are performed via implementation logic using `AMPUtils` helper functions.

Main Entry Points and Actors

- `fallback()`: Invoked by external callers, users, or authorized role holders to interact with the underlying implementation contract, verifying access rights through the `IAccessManager` (unless the target selector is configured as a pass-through method) before delegating the call.
- `receive()`: Invoked by external callers or contracts transferring native tokens with empty data, which verifies access permissions for the receive action before delegating execution to the implementation contract.

Code Diagram



✦ 1 of 4 Findings

contracts/AMPUtils.sol contracts/AccessManagedProxyBase.sol

An incompatible authority can permanently disable all non-pass-through proxy calls, including gated recovery paths

• Medium Risk

AMPUtils.setAccessManager only checks that the proposed authority address currently has code. It does not verify that the contract can return a valid `IAccessManager.canCall(address,address,bytes4)` response. After an authorized implementation call installs a reverting, malformed, non-implementing, or self-referential authority, `AccessManagedProxyBase._delegate` fails before `delegatecall` for every selector not present in `skipAc`. If the authority setter and implementation-upgrade entrypoints are also non-pass-through, neither can be reached to repair the configuration, permanently disabling those gated proxy operations. Compatibility validation, a staged transition, or an independently protected recovery mechanism is required.

Severity Note:

- Unrecoverability assumes authority-changing and upgrade selectors are not pass-through; a deployment that skipAc's those selectors can still repair.
- Harm is availability of gated calls; these contracts do not demonstrate an execute path that moves user funds.

✦ 2 of 4 Findings

contracts/AccessManagedProxyBase.sol contracts/AMPUtils.sol

Proxy mishandles delayed AccessManager operations, rejecting direct calls and substituting the AccessManager as caller on relays

• Medium Risk

`AccessManagedProxyBase._delegate` treats the `immediate` return value from `ACCESS_MANAGER().canCall` as complete authorization. For a role member with a nonzero OpenZeppelin AccessManager execution delay, `canCall(member, proxy, selector)` returns `immediate == false` even when the member is authorized to perform the operation after scheduling it. The proxy neither uses the returned delay nor consumes the scheduled operation. Consequently, a delayed role member calling the proxy directly is reverted with `AccessManagedUnauthorized` before the scheduled operation can be validated, leaving delayed proxied actions such as upgrades, vault-management actions, parameter changes, pauses, or recovery actions uncancellable through that path.

The alternative `AccessManager.schedule` followed by `AccessManager.execute(proxy, data)` path has different but unsafe semantics. The call arriving at the proxy originates from the AccessManager, rather than the account that scheduled or executed the operation. During an in-flight execution, `canCall(address(accessManager), proxy, selector)` takes AccessManager's self-caller execution-id validation path and returns an immediate authorization, so `_delegate` proceeds. The subsequent `delegatecall` preserves this substituted `msg.sender`: implementation code observes the AccessManager address, and the original authorized account is not recoverable from the forwarded calldata.

Accordingly, implementation logic based on `msg.sender` performs actions for the AccessManager rather than the originating account, including token pulls using `transferFrom(msg.sender, ...)`, per-account accounting, operator recording, or ownership assignment. Secondary checks using `AMPUtils.checkCanCall(msg.sender, customSelector)` likewise evaluate the AccessManager address under the in-flight execution-id path rather than evaluating the original caller's roles. Implementations that regard the AccessManager address as trusted can therefore be reached with `msg.sender` equal to that address by an authorized caller routing the call through `execute`. The proxy also does not implement the scheduled-operation consumption interface expected of an `IAccessManaged` target, so delayed-operation handling is not performed by the target.

Severity Note:

- Direct loss of user funds is not shown in this proxy; `transferFrom`, per-account accounting, and ownership assignment require implementations that key on `msg.sender`.
- During `execute`, `AMPUtils.checkCanCall(msg.sender, customSelector)` evaluates AccessManager via the in-flight execution id and will fail for a different custom selector rather than impersonate the original member's roles.

✦ 3 of 4 Findings

contracts/AMPUtils.sol

`AMPUtils.setPassThruMethods` overwrites the observability array without clearing the previous `skipAc` entries, leaving invisible access-control bypasses

• Low Risk

`AMPUtils` keeps two parallel structures for the pass-through configuration: `passThruMethods` (the array returned by `AccessManagedProxy.PASS_THRU_METHODS()`, used only for observability) and `skipAc` (the mapping actually consulted by `AccessManagedProxy._skipAc`, which is what `AccessManagedProxyBase._delegate` uses to decide whether the `ACCESS_MANAGER().canCall` gate is executed at all).

`setPassThruMethods` only adds entries to the mapping while completely replacing the array:

```
function setPassThruMethods(bytes4[] memory passThruMethods) internal {
    AccessManagedProxyStorage storage $ = AMPUtils.getAccessManagedProxyStorage();
    $.passThruMethods = new bytes4[](passThruMethods.length); // array fully replaced
    for (uint256 i; i < passThruMethods.length; ++i) {
        $.passThruMethods[i] = passThruMethods[i];
        $.skipAc[passThruMethods[i]] = true; // mapping only ever set to true
    }
    emit IAccessManagedProxy.PassThruMethodsChanged(passThruMethods);
}
```

Only `replacePassThruMethods` iterates the old array and resets `skipAc[old] = false` before calling `setPassThruMethods`. Both functions are `internal` members of a library that, per the design, is called from the implementation contract through admin wrapper functions, and nothing in `setPassThruMethods` restricts it to a first-time/virgin-storage configuration.

Consequences when `setPassThruMethods` is used on already-configured storage:

1. Every selector that was in the previous list but is omitted from the new list keeps `skipAc[selector] == true`. Such a selector permanently bypasses the whole authorization gate in `_delegate` (`_skipAc` returns true, so `canCall` is never queried), yet `PASS_THRU_METHODS()` no longer reports it. The bypass is therefore both live and invisible to off-chain observers and monitoring built on the documented observability surface.
2. The same desync is reachable at deployment time. `AccessManagedProxy`'s constructor runs the ERC-1967 initialization delegatecall (`_data`) before configuring the proxy storage:

```
constructor(..., bytes memory _data, IAccessManager accessManager, bytes4[] memory
passThruMethods)
    payable AccessManagedProxyBase(implementation, _data)
{
    AMPUtils.setAccessManager(accessManager);
    AMPUtils.setPassThruMethods(passThruMethods);
}
```

If the initializer invoked through `_data` configures pass-through selectors itself (a natural thing for an initializer that mirrors the admin setters), the constructor's subsequent `setPassThruMethods` replaces the array with the constructor argument while the initializer's selectors silently remain flagged in `skipAc`.

The result is an authorization bypass for arbitrary selectors (including state-changing or privileged implementation entry points that were legitimately pass-through at some earlier point in the contract's life) that cannot be detected from the contract's public configuration views, and that can only be removed by a subsequent `replacePassThruMethods` call that re-lists and then drops those exact selectors.

Severity Note:

- Medium/High harm assumes dropped pass-through selectors are or become security-sensitive privileged methods; AMPUtils does not demonstrate such methods or fund movement.
- Reachability assumes an implementation invokes `setPassThruMethods` on already-populated `skipAc` (admin wrapper or initializer plus constructor) rather than `replacePassThruMethods`.

✦ 4 of 4 Findings

contracts/AccessManagedProxyBase.sol contracts/AMPUtils.sol

Synthetic and "fake" selectors share the AccessManager selector namespace without domain separation

[• Info](#)

The framework injects three classes of 4-byte keys into the same `AccessManager` permission namespace that real external function selectors occupy (`allowedRoles[target][selector]`):

```
bytes4 internal constant RECEIVE_SELECTOR = bytes4(keccak256("receive"));
bytes4 internal constant FALLBACK_SELECTOR = bytes4(keccak256("fallback"));
```

```
function makeSelector(bytes memory something) internal pure returns (bytes4) {
    return bytes4(keccak256(something));
}
```

None of these values is domain-separated from ABI function selectors (which are also `bytes4(keccak256(signature))`), and `checkCanCall` queries the manager with them exactly as if they were real selectors:

```
(bool immediate, ) =
IAccessManagedProxy(payable(address(this))).ACCESS_MANAGER().canCall(user, address(this),
selector);
require(immediate, AccessManagedUnauthorized(user));
```

Because the manager keys permissions by the raw `bytes4`, two distinct concepts that hash to the same 4 bytes become a single permission. A fake selector produced by `makeSelector` that collides with a real external function selector means that the role assigned to the external function is automatically also granted for the internal privileged operation guarded by `checkCanCall`, and vice versa. The same applies to `RECEIVE_SELECTOR` / `FALLBACK_SELECTOR` colliding with a real implementation function: granting permission to receive plain ETH would also grant that function.

The search space is only 2^{32} , so a collision is not merely a remote accident: a party able to influence the naming of a permission string or of an implementation function can grind a preimage that aliases a permissive selector (for example one assigned to `PUBLIC_ROLE`) onto a privileged permission checked by `checkCanCall`. Nothing in the library or proxy detects or prevents the aliasing, and the collision is not visible in `PASS_THRU_METHODS()` or in the manager's configuration, which only stores opaque `bytes4` keys.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.